



CELEBRATING 50 YEARS

Dilip Kumar Maripuri
Computer Applications





Session : Binary Search Trees, Insertion into a BST



Data Structures

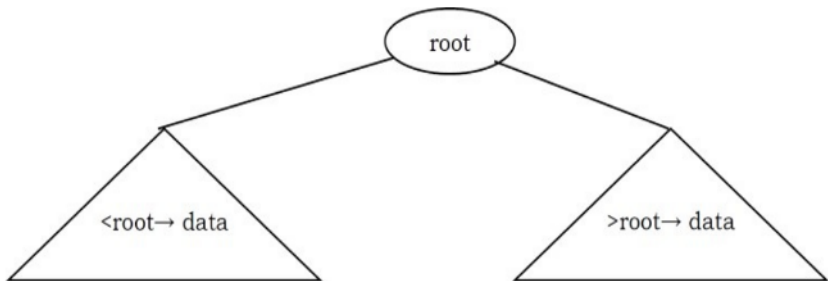
Binary Search Trees

- ▶ A **Binary Search Tree (BST)** is a node-based binary tree data structure where each node follows the BST property:
 - ▶ The **left subtree** of a node contains only nodes with values **less than** the node's key.
 - ▶ The **right subtree** of a node contains only nodes with values **greater than** the node's key.
 - ▶ The left and right subtrees must also be **binary search trees**.
- ▶ BSTs provide a structured and efficient way of organizing, storing, and retrieving data dynamically.

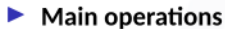


Data Structures

Binary Search Trees







- ▶ Find / Find Minimum / Find Maximum element in binary search trees
- ▶ Inserting an element in binary search trees
- ▶ Deleting an element from binary search trees

► **Auxiliary operations**

- ▶ Checking whether the given tree is a binary search tree or not
- ▶ Finding k^{th} -smallest element in tree
- ▶ Sorting the elements of binary search tree amongst others



Data Structures

Binary Search Trees

► Points to Note

- Since root data is always between left subtree data and right subtree data, performing **inorder traversal** on a binary search tree produces a **sorted list**.
- If we are searching for an element and if the left subtree root data is less than the element we want to search, then skip it. The same applies to the right subtree.
 - Because of this, binary search trees take less time for searching an element than regular binary trees.
 - In other words, the binary search trees consider either left or right subtrees for searching an element but not both.



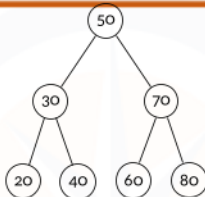


Data Structures

Inorder Predecessor

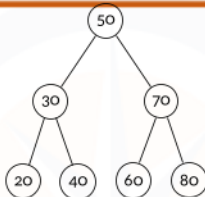


- ▶ The inorder predecessor of a node x is defined as follows
 - ▶ If x has a left subtree, the predecessor is the maximum value node in that subtree.
 - ▶ This can be found by traversing to the rightmost node of the left subtree.
 - ▶ If x does not have a left subtree:
 - ▶ If x is a **right child** of its parent, then the parent is the predecessor.
 - ▶ If x is a **left child**, traverse up using parent pointers until you find a node that is a right child of its parent; that parent will be the predecessor.
 - ▶ If you not find any such node then , there exists no predecessor

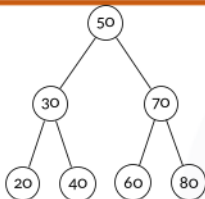


Node	Inorder Predecessor	How It Was Found
20	None	No left subtree, no smaller node
30	20	Rightmost node in the left subtree
40	30	No left subtree, found by moving up to the nearest smaller ancestor
50	40	Rightmost node in the left subtree
60	50	No left subtree, found by moving up to the nearest smaller ancestor
70	60	Rightmost node in the left subtree
80	70	No left subtree, found by moving up to the nearest smaller ancestor





Node	Inorder Successor	How It Was Found
20	30	No right subtree, found by moving up to the nearest larger ancestor
30	40	Leftmost node in the right subtree
40	50	No right subtree, found by moving up to the nearest larger ancestor
50	60	Leftmost node in the right subtree
60	70	No right subtree, found by moving up to the nearest larger ancestor
70	80	Leftmost node in the right subtree
80	None	No right subtree, no larger node



Inorder Traversal: 20, 30, 40, 50, 60, 70, 80

Node	Inorder Predecessor	Inorder Successor
20	None	30
30	20	40
40	30	50
50	40	60
60	50	70
70	60	80
80	70	None

A word cloud shaped like the letter 'A' containing various terms related to agriculture and food systems. The words include: **QUESTIONS**, **Food Systems**, **Food Security**, **Food Access**, **Food Quality**, **Food Safety**, **Food Waste**, **Food Policy**, **Food Justice**, **Food Sovereignty**, **Food Equity**, **Food Culture**, **Food Identity**, **Food Community**, **Food Movement**, **Food Activism**, **Food Education**, **Food Research**, **Food Innovation**, **Food Entrepreneurship**, **Food Social Enterprise**, **Food Non-Profit**, **Food Cooperative**, **Food Collective**, **Food Alliance**, **Food Network**, **Food Hub**, **Food Bank**, **Food Shelf**, **Food Store**, **Food Market**, **Food Fair**, **Food Festival**, **Food Event**, **Food Program**, **Food Project**, **Food Initiative**, **Food Campaign**, **Food Movement**, **Food Activism**, **Food Education**, **Food Research**, **Food Innovation**, **Food Entrepreneurship**, **Food Social Enterprise**, **Food Non-Profit**, **Food Cooperative**, **Food Collective**, **Food Alliance**, **Food Network**, **Food Hub**, **Food Bank**, **Food Shelf**, **Food Store**, **Food Market**, **Food Fair**, **Food Festival**, **Food Event**, **Food Program**, **Food Project**, **Food Initiative**, **Food Campaign**.

Algorithm 1 Create a New Node in BST

```

1: Start
2: root  $\leftarrow$  (struct node*) malloc(sizeof(struct node))
3: Assign value to root  $\rightarrow$  data
4: Assign NULL to root  $\rightarrow$  left
5: Assign NULL to root  $\rightarrow$  right
6: return root

```




Data Structures

BST Construction



► Step 2: Recursive Cases

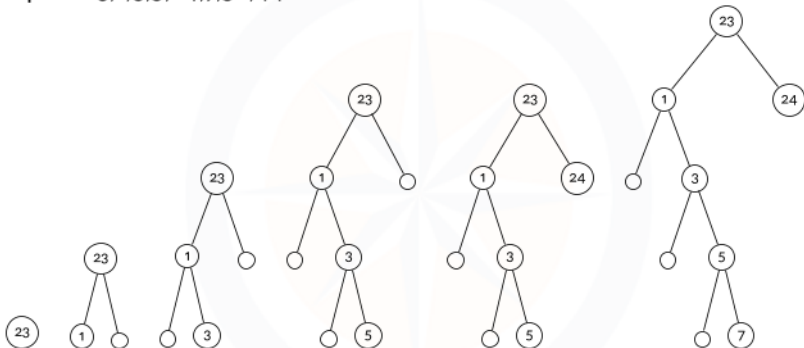
- If the value to be inserted is **less than** the root's data:
 - Recursively call `insert()` on the **left subtree**.
 - Assign the returned value to `root->left`.
- Else if the value is **greater than or equal to** the root's data:
 - Recursively call `insert()` on the **right subtree**.
 - Assign the returned value to `root->right`.

► Step 3: Return the Root

- After inserting the node, return the `root` to maintain the BST structure.



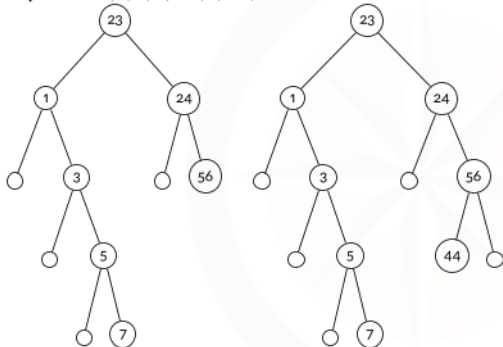
Input : 23,1,3,5,24,7,56,44





Data Structures

Input : 23,1,3,5,24,7,56,44





```

1: function INSERT(root, value)
2:   if root = NULL then
3:     root  $\leftarrow$  CreateNode(value)
4:     return root
5:   end if
6:   if value < root.data then
7:     root.left  $\leftarrow$  INSERT(root.left, value)
8:   else
9:     root.right  $\leftarrow$  INSERT(root.right, value)
10:  end if
11:  return root
12: end function

```



```
1 typedef struct MyNode {
2     int data;
3     struct MyNode *left, *right;
4 } *NODE;
5
6 NODE createNode(int value) {
7     NODE newNode = (NODE)malloc(sizeof(struct
8         MyNode));
9     if (newNode == NULL) {
10         printf("Memory allocation failed!\n");
11         return NULL;
12     }
13     newNode->data = value;
14     newNode->left = newNode->right = NULL;
15     return newNode;
16 }
```



```
1  NODE insert(NODE root, int value) {  
2      if (root == NULL)  
3          return createNode(value);  
4  
5      if (value < root->data)  
6          root->left = insert(root->left, value);  
7      else  
8          root->right = insert(root->right, value)  
9          ;  
10     return root;  
11 }
```



```
1 int main() {  
2     NODE root = NULL;  
3     root = insert(root, 20);  
4     .....  
5     // Use inorder traversal to check if BST was  
        constructed Properly  
6 }
```




Thank You

Dilip Kumar Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu
8073212026